THE MACHINE LEARNING CANVAS

Designed for: EnergyForecast AI (WTI Crude Oil Forecasting)

Designed by: Kai Shahram

Course: AASD 4016 / AASD 4017

DECISIONS

How are predictions used to make decisions that provide the proposed value to the end-user?

• Predicted next-day WTI price supports short-term decisions: planning, budgeting, procurement, risk assessment, market analysis
• Users compare the forecast against current market price for added context
• Model provides decision support only - it does not automatically execute trading or operational decisions
• This forecasting system is also intended to be used as part of a larger "Optimal Energy Resource Planning" project

ML TASK

Input, output to predict, type of problem.

• Input: most recent 60 days x 26 features -> 60x26 matrix
• Output: predicted next-day WTI crude oil price (USD/barrel)
• Type: Supervised time-series regression - continuous value from sequential historical data

VALUE PROPOSITIONS

What are we trying to do for the end-user(s) of the predictive system? What objectives are we serving?

• Provide an accessible, automated next-day oil-price forecasting service
• System auto-collects market data, engineers features, builds the 60-day input, and generates the forecast
• Users don't need to manually collect/prepare data or run the LSTM model themselves
• Broader goal: timely quantitative information to support short-term energy/market decisions

DATA SOURCES

Which raw data sources can we use (internal and external)?

• FRED API: WTI crude oil price, natural gas price, U.S. Dollar Index, VIX
• yfinance: Gold market data
• Historical data builds model features; recent observations feed live predictions

COLLECTING DATA

How do we get new data to learn from (inputs and outputs)?

• Live prediction auto-retrieves ~500 days of recent data from FRED and yfinance
• Extra history needed since some engineered features use rolling windows up to 60 days
• For retraining: new observations get added to the historical dataset; once actuals arrive for a predicted day, they become target values for evaluation/retraining

MAKING PREDICTIONS

When do we make predictions on new inputs? How long do we have to featurize a new input and make a prediction?

• On-demand, when the user requests a forecast via the Streamlit app
• Pipeline: fetch data -> engineer 26 features -> latest 60 complete rows -> 60x26 matrix -> FastAPI -> LSTM predicts -> result returned
• Meant to be interactive/near-real-time; can be slower if free-tier Render/Streamlit services need to wake up

OFFLINE EVALUATION

Methods and metrics to evaluate the system before deployment.

• Evaluated on held-out historical data not used in training
• 80/20 chronological split (not random) to preserve time-series order
• MAPE: 2.97% | R^2: 0.928 | MAE: ~$2.35

FEATURES

Input representations extracted from raw data sources.

• 26 engineered features: market levels (Oil, Gold, Nat Gas, USD Index, VIX); lags (1/5/10/21-day); momentum (7/21-day)
• Rolling stats (7/21/60-day mean & std dev), volatility, log returns, Gold/Oil ratio, VIX spike indicator, calendar features
• Same feature pipeline used for training and live prediction - avoids train/serve skew

BUILDING MODELS

When do we create/update models with new training data? How long do we have to featurize training inputs and create a model?

• LSTM trained offline on ~20 years of daily data (2006-2026)
• Architecture: LSTM(128) -> Dropout(0.2) -> LSTM(64) -> Dropout(0.2) -> Dense(32) -> Dense(1)
• TensorFlow/Keras, Adam optimizer, MSE loss, MinMax scaling, EarlyStopping, ReduceLROnPlateau
• No auto-retraining currently; retraining done offline as new data accumulates or performance degrades

LIVE EVALUATION AND MONITORING

Methods and metrics to evaluate the system after deployment, and to quantify value creation.

• Compare predicted vs. actual WTI prices once actuals become available; track MAE, RMSE, MAPE, R^2 over time
• Technical monitoring: API availability, /health status, data-source availability, missing observations, failed predictions, response times, Render/Streamlit uptime
• Rising errors or shifting input data characteristics signal drift -> triggers retraining/reevaluation